GravityLens: Problem Definition — Deep Dive

Why understanding your AWS architecture matters more than you think

This document explores the real-world problems that GravityLens solves, with specific scenarios, pain points, and business impact.

Table of Contents

Problem Landscape

Organizations using AWS at scale face a universal challenge:

As infrastructure grows, visibility decreases.

This happens because:

1. **AWS is service-oriented** — there are 200+ services, each with its own console, API, and monitoring
2. **Resources are deeply nested** — VPCs contain subnets contain EC2 instances, each with security groups, IAM roles, etc.
3. **Relationships are implicit** — that Lambda can access that S3 bucket because of its IAM role, but this relationship exists nowhere as a first-class entity in AWS
4. **Change is constant** — infrastructure evolves daily, but there's no built-in way to track that evolution

The result: **Infrastructure becomes a black box.**

Scenario 1: The 3 AM Production Crisis

Context

You're the on-call SRE. It's 3:14 AM. Your phone explodes.

PagerDuty Alert: *"API response time: 8000ms (threshold: 200ms)"*

Your team is losing customers every second. Your manager is asking: *"What happened?"*

Current Workflow (Without GravityLens)

3:15 AM — You wake up, check the alert

- Alert shows: API Gateway latency is high
- You don't know if the issue is in API Gateway or downstream

3:17 AM — Open AWS Console, navigate to API Gateway

- You see: Requests are coming in, but they're timing out
- You check CloudWatch metrics (APICallCount is normal, but errors are high)
- **Question:** Is the issue in API Gateway, or in what it calls?

3:22 AM — Open Lambda console

- You find which Lambda function is invoked by this API Gateway route
- You check CloudWatch Logs
- You see: Lambda is failing with a timeout error
- **Question:** Why is Lambda timing out? Is it Lambda itself, or something Lambda calls?

3:28 AM — Check Lambda configuration

- You see: The Lambda has a 30-second timeout
- CloudWatch logs show it's taking 35 seconds to complete
- It's calling an RDS database
- **Question:** Is RDS slow?

3:33 AM — Open RDS console

- You check RDS Performance Insights
- You see: CPU is at 95%, connection count is at max (100 connections)
- **Question:** Why so many connections? Which services are connecting?

3:38 AM — Dig deeper

- You check CloudTrail to see what changed recently
- CloudTrail shows 500+ API calls in the last 2 hours
- You manually parse them to find what changed

- You find: A new Lambda function was deployed 2 hours ago
- **Hypothesis:** The new Lambda is opening connections to RDS and not closing them

3:45 AM — Verify hypothesis

- You check if the new Lambda is using connection pooling
- It's not. It opens a new connection for every invocation
- With 1000 requests/second, that's 1000 connections/second
- RDS max is 100 connections
- **Root cause found:** Lambda not using connection pooling

3:50 AM — Fix deployed

- The team deploys a fix (Lambda now uses connection pooling)
- RDS connections drop to 10
- API response time normalizes

Total time to resolution: 35 minutes

With GravityLens

3:15 AM — Open GravityLens dashboard

- Instant view: API Gateway → Lambda → RDS
- The graph shows: Lambda and RDS are connected
- Performance overlay shows: Lambda is healthy, but RDS CPU is at 95%
- **Immediate insight:** The issue is downstream from Lambda

3:16 AM — Click on RDS node

- Detail panel shows: 100 active connections
- You check the History tab to see what changed
- You see: A new Lambda function was added 2 hours ago
- You compare the previous version (no new Lambda) with the current version
- **Insight:** New Lambda is new since the outage

3:18 AM — Watch the Replay

- You watch the animation of the new Lambda being added
- The animation shows: New Lambda is connected to RDS
- You immediately check that Lambda's code
- It's missing connection pooling

3:22 AM — Fix deployed

Total time to resolution: 7 minutes

The Difference

Stage	Without GravityLens	With GravityLens
Understanding what happened	25 minutes	2 minutes
Deploying fix	10 minutes	5 minutes
Total MTTR	**35 minutes**	**7 minutes**
Customer impact	35 minutes × 1000 req/s = 2M failed requests	7 minutes × 1000 req/s = 420k failed requests
Revenue impact	~$5,000+ lost	~$1,000 lost

Scenario 2: The Onboarding Nightmare

Context

A new engineer (Alex) joins your team. Their first task: understand the production architecture.

Your team says: *"It should take a day. You'll get up to speed quickly."*

Spoiler: It takes a week.

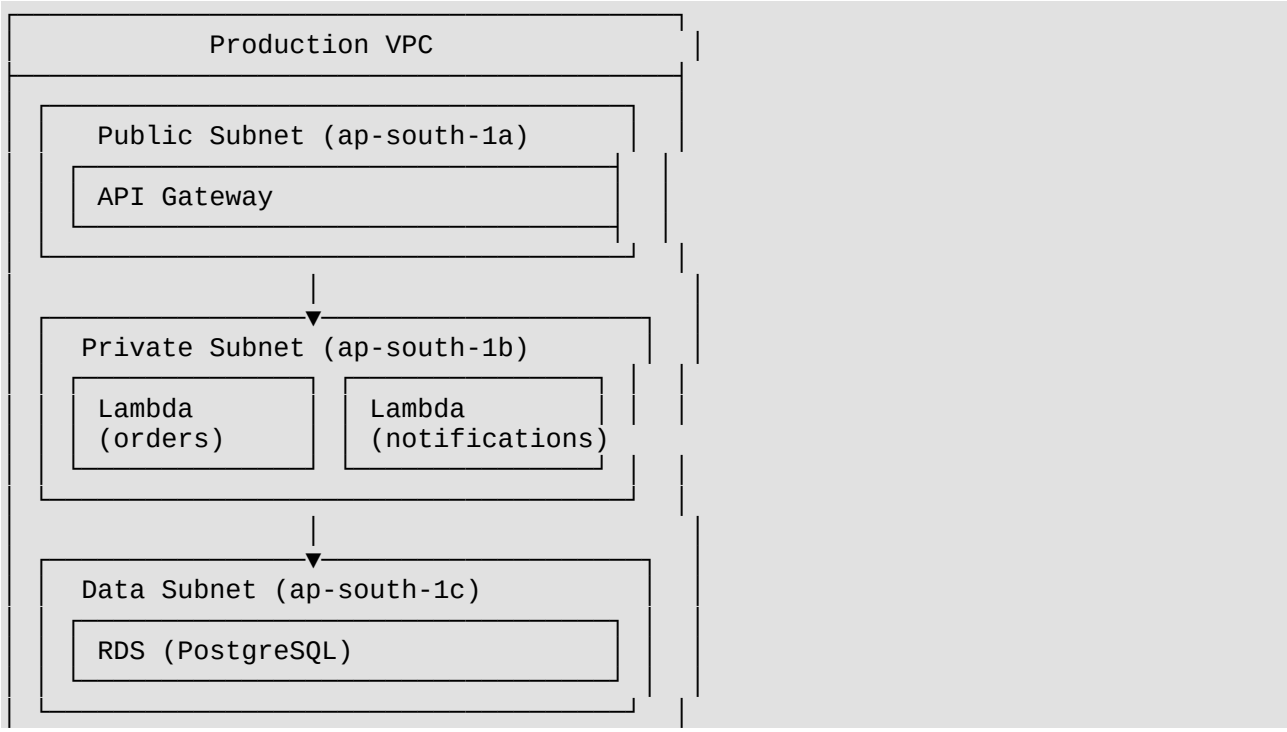
Current Workflow (Without GravityLens)

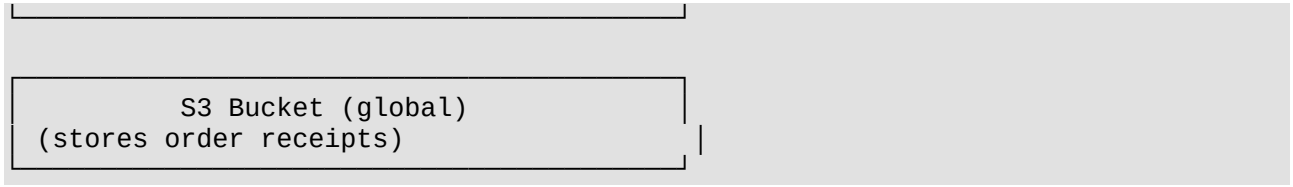
Day 1, Morning — Alex gets access to AWS Console

They ask: *"Can someone show me the architecture?"*

You say: *"Sure, let me draw a quick diagram"*

You spend 30 minutes drawing in Lucidchart:





Alex looks at this and says: *"Wait, how does API Gateway talk to Lambda? What are the security groups? What IAM roles do they have? How does Lambda authenticate to RDS?"*

Day 1, Afternoon — Alex starts exploring AWS Console

They navigate:

1. API Gateway console → find the integration → see which Lambda is invoked
2. Lambda console → find the function → check VPC configuration → see security groups
3. VPC console → check security group rules
4. EC2 console → find the security group → see what ports are open
5. IAM console → find the Lambda's role → see what permissions it has

By 5 PM, Alex has manually traced the path: API Gateway → Lambda → RDS

But they still don't know:

- Which Lambda talks to S3?
- Does Lambda connect to any external services?
- What happens if RDS fails?
- How are notifications sent?
- Is there any caching layer?

Day 2 — Alex asks more questions

They ask: *"Does anything else call that RDS database?"*

You have to manually check each Lambda, each service, trace the relationships.

Day 3 — Alex asks about SQS

They noticed an SQS queue in the console.

They ask: *"What's this queue for? Who produces messages? Who consumes them?"*

You have to trace through Lambda functions to find the producers and consumers.

Day 4 — Alex understands the basics

But there's one more Lambda function you forgot to mention in the diagram.

It connects to an external API that you've outsourced to a third party.

Alex discovers this by accident in the code.

Your diagram was already out of date.

Day 5 — Alex is finally productive

It's taken them a full week to understand what should have taken a day.

With GravityLens

Day 1, 9 AM — Alex joins the standup

You say: *"Your task today is to understand the production architecture"*

You open GravityLens on the projector:

```
├─ API Gateway
│  └─→ Lambda (orders)
│      ├─→ RDS
│      └─→ S3
│
├─ Lambda (notifications)
│  ├─→ RDS
│  └─→ SES
│
└─ SQS
   └─→ Lambda (async-processor)
       └─→ RDS
```

Alex sees the entire architecture at a glance.

They click on nodes and ask questions:

- Click Lambda (orders) → see what it does, what resources it accesses
- Click RDS → see which Lambdas depend on it
- Click S3 → see who writes to it

Day 1, 10 AM — Alex watches the Replay

You show them the "Replay" feature:

"This is how the architecture evolved over time. Watch."

They watch a 30-second animation showing how each service was added, modified, and removed over the past month.

They see exactly when the external API integration was added (day 15 of the month).

Day 1, 12 PM — Alex has a comprehensive understanding

They've seen:

- Current architecture (interactive graph)
- How it evolved (replay animation)
- Each service and its purpose (detail panels)
- Relationships between services (edges)

Day 2 — Alex is productive

They can navigate around the codebase, understand where services connect, and contribute meaningfully.

The Difference

Stage	Without GravityLens	With GravityLens
Understand basic architecture	1 day (manual exploration)	30 minutes (visual + replay)
Find all services and relationships	3 days (manual tracing)	30 minutes (graph is complete)
Understand architecture evolution	1 day (reading code history)	10 minutes (watch replay)
Time to productivity	5 days	1 day
Cost (if $50/hour)	$2,000	$400

Scenario 3: The Hidden Cost Problem

Context

Your AWS bill has been growing 15% month-over-month.

Your CFO asks: *"Why are we spending $50,000 more per month than last month? Where is the money going?"*

You don't have a good answer.

Current Workflow (Without GravityLens)

You go to the AWS Cost Explorer:

1. **High-level view shows:** $50k increase in "Data Transfer" costs
2. **You dig deeper:** Most of it is cross-availability zone (cross-AZ) data transfer
3. **Question:** Which services are communicating across AZs? This is expensive and unnecessary.

You start manually checking:

- Is RDS in AZ-1 but Lambda in AZ-2? (yes)
- Is there caching that could reduce cross-AZ traffic? (you don't know)
- Which specific connections are cross-AZ? (AWS doesn't show this visually)

You spend hours tracing through VPC Flow Logs or using third-party tools.

By the time you identify the problem (RDS replica not in the same AZ as Lambda), a week has passed.

With GravityLens

1. Open GravityLens dashboard

You see the graph with each node labeled with its AZ:

```
Subnet (ap-south-1a)          Subnet (ap-south-1b)
├─ Lambda                     ├─ RDS (replica)
└─ [Data transfer: 50GB/hour] └─ [idle]

Connection: Lambda (ap-south-1a) → RDS (ap-south-1b)
⚠ Cross-AZ data transfer: 50GB/hour
💰 Monthly cost: $2,500
```

2. Immediate insight

Lambda and RDS are in different AZs. This is why cross-AZ data transfer is high.

The fix is obvious: provision an RDS read replica in the same AZ as Lambda.

Estimated savings: $2,500/month

3. Verify by comparing versions

You check the History tab and see when this problem started.

6 weeks ago, RDS was in the same AZ as Lambda.

Someone deployed a new Lambda in a different AZ but didn't provision a replica.

Decision time: Provision replica, save $2,500/month

The Difference

Stage	Without GravityLens	With GravityLens
Identify the problem	1 week (manual log analysis)	5 minutes (visual overlay)
Understand the root cause	3 days (VPC Flow Logs)	Immediate (graph shows AZ labels)
Decide on fix	1 day (analysis)	Immediate (fix is obvious)
Yearly savings	–	$30,000

Scenario 4: The Security Audit

Context

Your company is preparing for an SOC 2 audit.

The auditor asks: *"Show me all the ways external requests reach your critical databases. Prove they're secure."*

Current Workflow (Without GravityLens)

You manually trace through:

1. **Find all ways to reach RDS:**
    - Via Lambda? (check Lambda's security group and IAM role)
    - Via EC2? (check EC2's security group)

- Via third-party service? (check RDS security group inbound rules)

2. **For each path, verify security:**
    - Is there a VPN?
    - Is there encryption?
    - Are credentials rotated?
    - Is there an audit trail?
3. **Create a document** showing all these paths

This takes 2-3 days of manual work.

The document is 50+ pages of screenshots and notes.

Problem: Next month, when architecture changes, this document is out of date.

With GravityLens

1. Open GravityLens

You filter the graph to show only paths to RDS:

```
        API Gateway
            │
           Lambda (auth)
            │
            RDS ◄── audit critical database
```

2. Click each connection

For each edge, you see:

```
Lambda → RDS
├─ VPC: in same VPC
├─ Security Group: inbound port 5432 restricted to Lambda's security group
├─ IAM Role: RDS access policy attached
├─ Encryption: TLS 1.2 enabled
├─ Audit: all queries logged to CloudTrail
└─ Last verified: 2026-06-15 10:30
```

3. Generate an audit report

GravityLens generates a PDF report showing:

- All paths to critical resources
- Security configuration for each path
- Last verified date for each configuration

4. Share the report

Auditor is satisfied. Next month, you re-run the audit and see if anything changed.

The Difference

Stage	Without GravityLens	With GravityLens
Manual tracing	2 days	N/A
Creating documentation	1 day	30 minutes (auto-generated)
Keeping documentation updated	Manual + error-prone	Automatic + always current
Audit confidence	Low (docs might be out of date)	High (generated from live infra)

Scenario 5: The Accidental Outage

Context

A junior engineer is removing old infrastructure.

They delete a security group.

30 seconds later: production is down.

The question: *"Which service did we break?"*

Current Workflow (Without GravityLens)

1. **Check CloudTrail** — find the delete operation
2. **Find the security group ID** — it's sg-012345678
3. **Search through console** — which resources were using this security group?
4. **Manual detective work** — check each EC2 instance, Lambda, RDS, etc.
5. **Finally find it** — API Gateway was using this security group
6. **Restore the security group** — manually re-create it

This takes 15-20 minutes. In that time, thousands of requests fail.

With GravityLens

Immediate view: You look at the graph and see:

```
Deleted: Security Group (sg-012345678)

This security group was used by:
├─ API Gateway (production)
│  └─ Lambda (orders)
│     └─ RDS
└─ Lambda (webhook)
```

Instant context: You know exactly what broke.

You restore the security group immediately.

Alternatively, you could have prevented this entirely — GravityLens could have warned:

"This security group is used by 2 critical services. Are you sure you want to delete it?"

The Difference

Stage	Without GravityLens	With GravityLens
Identify what broke	10 minutes	30 seconds
Restore service	5 minutes	1 minute
Total downtime	**15 minutes**	**1 minute**
Requests served	0 (service is down)	~95% (some traffic served)

Root Cause Analysis

Why do all these problems exist?

Problem 1: AWS Console is Service-Centric

The AWS Console organizes around services:

- EC2 console
- Lambda console
- RDS console
- VPC console
- S3 console
- etc.

This is great for managing individual resources, but terrible for understanding your architecture.

What you need: A service-agnostic view that shows relationships

Problem 2: Relationships are Implicit

In AWS, relationships are encoded in multiple places:

- VPC associations (where is this Lambda?)
- Security groups (which ports are open?)
- IAM roles (what resources can this service access?)
- Explicit integrations (API Gateway → Lambda)
- Implicit integrations (Lambda code calls S3, but this isn't tracked by AWS)

AWS doesn't surface relationships as first-class entities. You have to manually piece them together.

What you need: A tool that extracts relationships from across AWS

Problem 3: Change Tracking is an Afterthought

AWS provides:

- CloudTrail (audit log of API calls)
- CloudWatch (metrics and logs)
- AWS Config (configuration changes)

But these don't answer: *"What did my infrastructure look like yesterday?"*

You have to reconstruct this manually or use a third-party tool.

What you need: Automatic snapshots of your infrastructure

Problem 4: Time to Understanding is High

Every problem requires:

1. Manual navigation through multiple consoles
2. Manual correlation between services
3. Manual reconstruction of relationships
4. Manual comparison of configurations over time

Each step is error-prone and time-consuming.

What you need: Instant visual understanding of your architecture

---

Business Impact Summary

Quantified Costs

Problem	**Time Cost**	**Financial Cost**	**Frequency**
Incident response (avg)	1 hour	$5,000	2x/month
Onboarding (per engineer)	5 days	$2,000	2x/year
Hidden costs (cross-AZ traffic)	–	$30,000/year	ongoing
Security audits	3 days	$1,500	2x/year
Accidental service degradation	15 min	$500	4x/year
Total annual cost	**100 days**	**$100,000+**	–

With GravityLens

Problem	**Time Saved**	**Financial Impact**	**Frequency**
Incident response (avg)	50 min/incident	$4,000	2x/month

Problem	Time Saved	Financial Impact	Frequency
Onboarding (per engineer)	4 days	$1,600	2x/year
Hidden costs (identified & fixed)	–	$30,000	1x
Security audits	2 days	$1,000	2x/year
Accidental degradation (prevented)	14 min/incident	$400	4x/year
Total annual impact	**50 days saved**	**$50,000+ saved**	–

ROI Calculation

Assumptions:

- Engineering labor: $100/hour
- Team size: 5 people
- GravityLens cost: $50/month = $600/year

Annual benefit:

- 50 days saved × 8 hours × $100 = $40,000
- Cost reductions = $30,000+
- **Total benefit: $70,000+**

ROI: 11,667% ($70,000 / $600)

Conclusion

The problems GravityLens solves are:

1. **Visibility** — see your entire architecture at a glance
2. **Change tracking** — understand what changed and why
3. **Relationships** — see which services depend on each other
4. **Time to understanding** — instant comprehension instead of hours of investigation

These problems cost organizations hundreds of thousands of dollars annually in lost productivity, incident response time, and hidden costs.

GravityLens solves all four with a single, elegant solution: **Show the infrastructure, not the services. Show relationships, not configurations. Show changes as stories, not as lists.**

Next Steps:

- Read the main README.md for the full GravityLens story

- Read ARCHITECTURE.md for technical details
- Read API_ENDPOINTS.md for API documentation